

---

# Optimizing Semantic Top-k Operators with Listwise Prompting

*Jason Shin, \*Jiwon Chang, Fatemeh Nargesian*  
*University of Rochester*

---

# Semantic operators extend SQL with NL

```
SELECT * FROM my_documents WHERE “...” GROUP  
BY “...” ORDER BY “...” LIMIT k;
```

“I can convey  
**semantic intent**  
in my queries!”



“I can query  
**unstructured or**  
**multimodal data**  
easily!”



# Semantic top-k

```
SELECT title, abstract FROM  
papers_db ORDER BY "The paper  
is relevant to my research  
studying...for...by applying  
technique..." LIMIT 10;
```

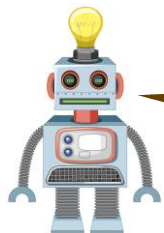


shutterstock.com • 2340757127



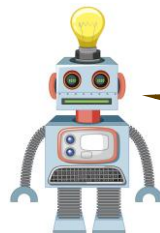
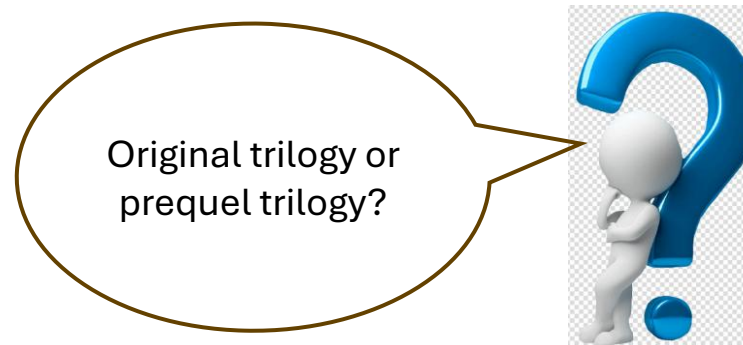
# LLM prompting paradigms

## Pointwise Prompting



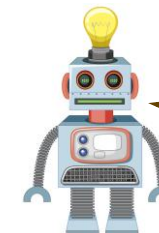
5 out of 7

## Pairwise Prompting



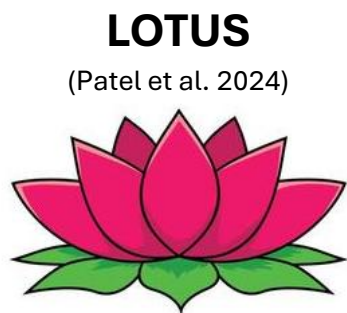
original

## Listwise Prompting



Fast & Furious,  
Fast X, 2 Fast 2  
Furious...

# Prior art: Pairwise prompting + quickselect

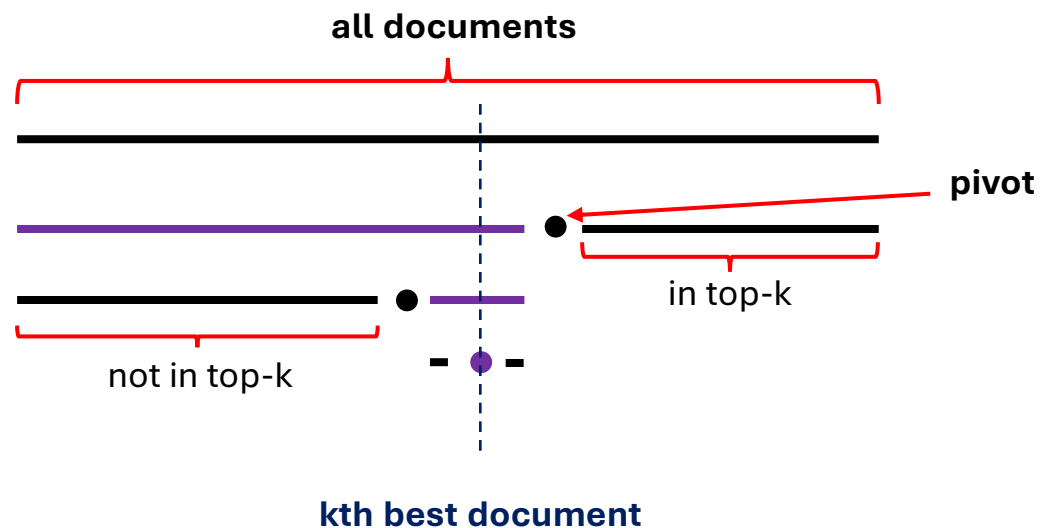


ELO  
mergesort  
priority queue

quickselect  
quicksort

pairwise  
comparison  
aggregation

# LOTUS top-k quickselect overview



**Prompt:** “Rank the following two documents based on their relevance to the user’s query.

User query: {user\_query}

A: {doc\_A} B: {doc\_B}

**LLM answer:** B, A

# Listwise prompting is more efficient

**Pairwise Prompting Tokens / Comparison**

$2 (\text{Document Size} + 1)$

**Listwise Prompting Tokens / Comparison**

$\frac{\text{Document Size} + 1}{\log_2(\text{List Size})}$



logarithmically more  
information per token!

# Problem: Listwise prompting is biased.

## Positive positional bias:

LLM's ordering correlated with the ordering of documents in prompt.

(Tang et al. 2024; Chao et al. 2024)





# Fine-tuned LLM rankers eliminate positive positional bias.

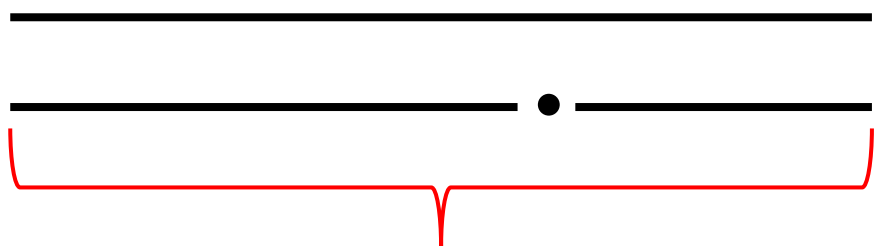
**RankZephyr**

(Pradeep et al. 2023)



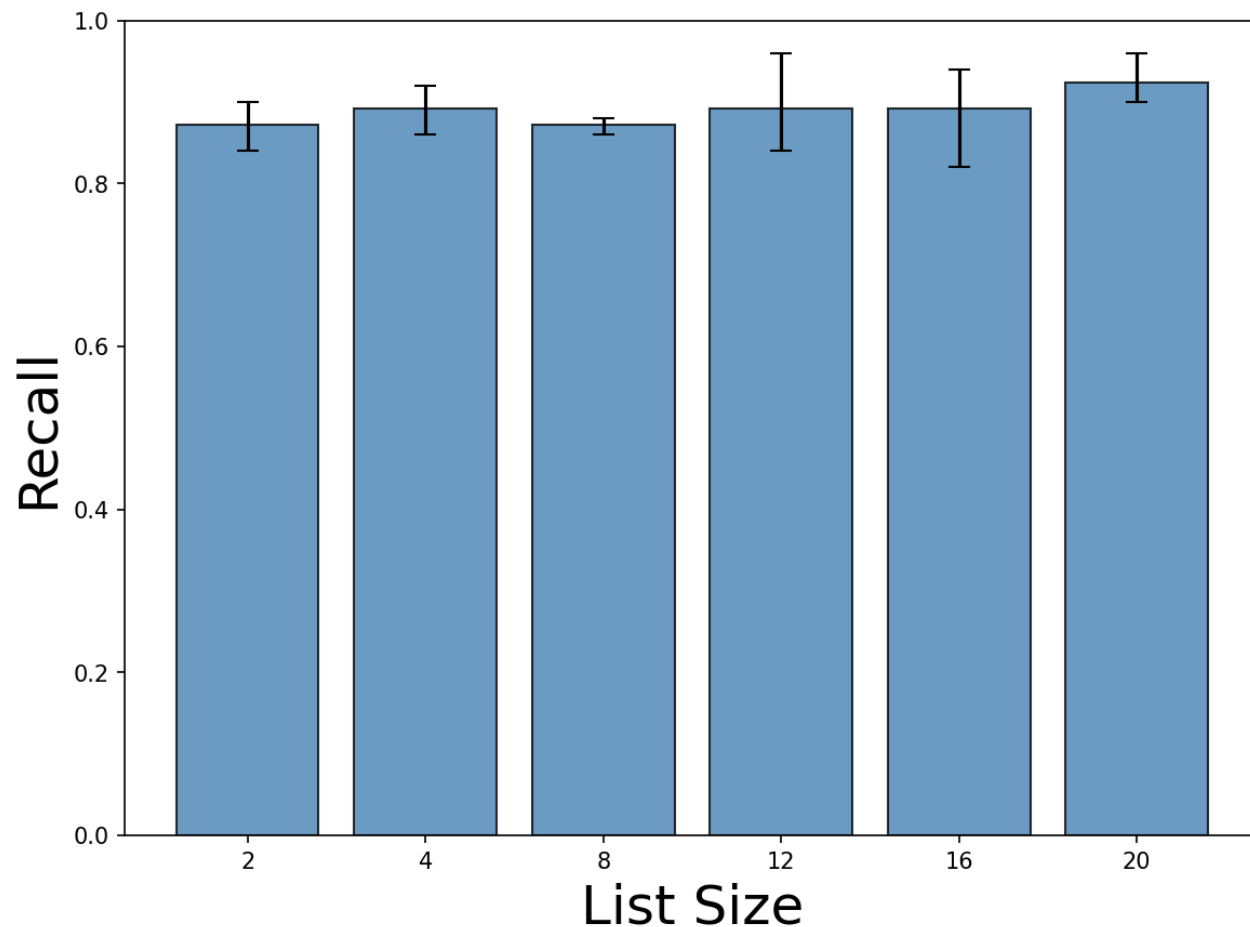
Lists of size up to 20,  
no problem!

# Listwise quickselect halves latency at no cost to accuracy.



bucket sort with listwise LLM calls

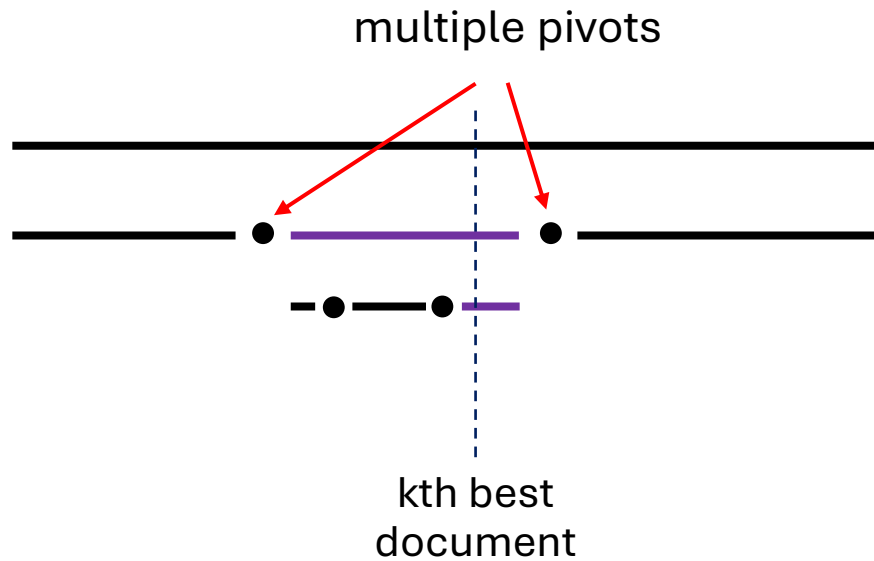
~46% Latency  
Reduction



**But we can do  
better!**



# Listwise multi-pivot quickselect



**Prompt:** Rank the following.

Doc 1: {pivot document 1}

Doc 2: {pivot document 2}

Doc 3: {non-pivot 1}

⋮

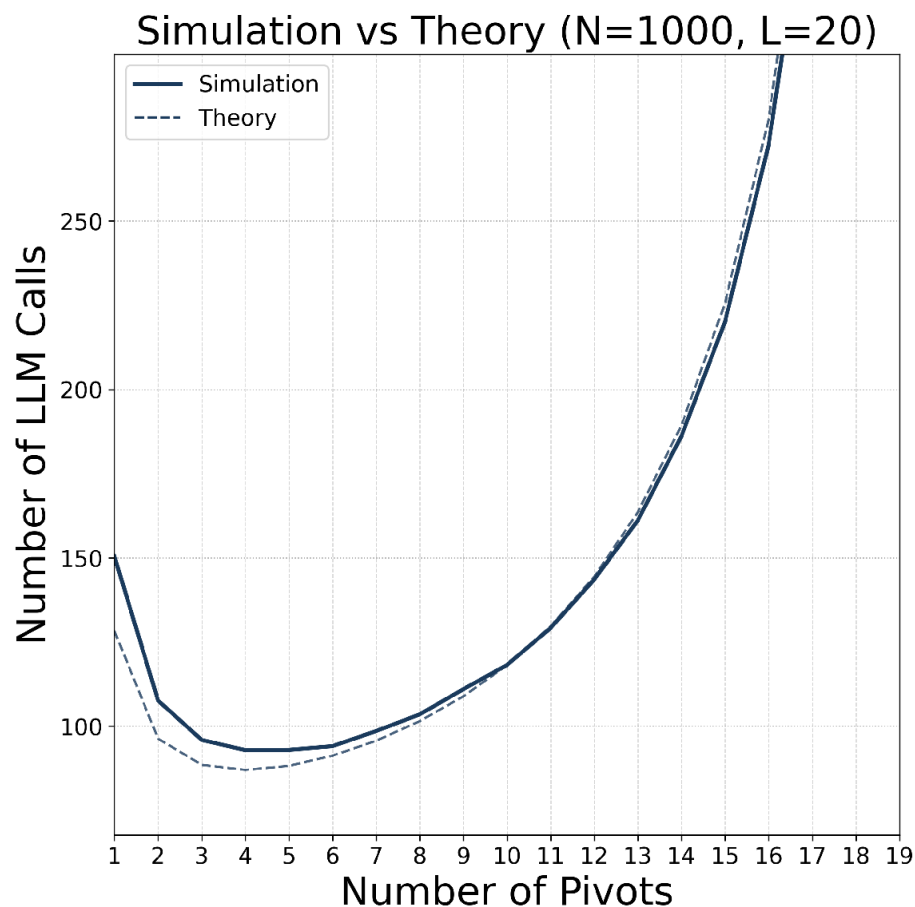
Doc 20: {non-pivot 18}

---

# Trade-off between more & less pivots

- **More pivots:** Reduce problem size faster; fewer total rounds.
- **Less pivots:** More non-pivot elements in each LLM call; faster bucket sort.

# Model for pivot count tuning



$$\text{LLM Calls}_k(n) = \frac{P + 1}{\underbrace{(W - P)}_{\text{trade-off between more \& less pivots}} \underbrace{\left( P - 1 + \left( \frac{K}{N} \right)^{P+1} + \left( 1 - \frac{K}{N} \right)^{P+1} \right)}_{\text{trade-off between more \& less pivots}}} \cdot \overset{\text{linear select}}{N}$$

**And there's  
more!**



---

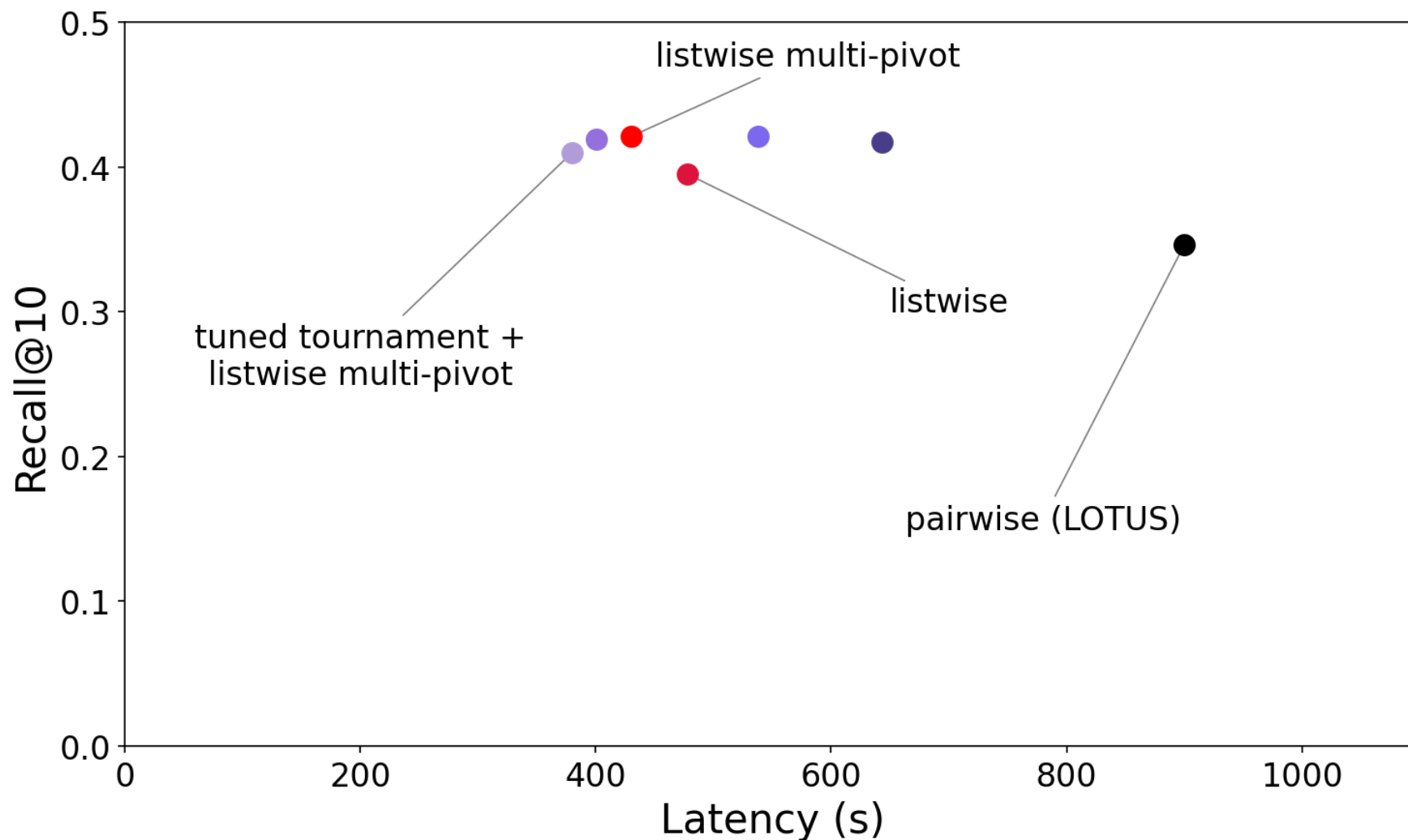
# Quickselect is Las Vegas; What about Monte Carlo approaches?

## Listwise Tournament filter

- Randomly shuffle elements into bins of 20 documents
- Listwise sort each bin
- Let the top-few in each bin survive



# End-to-end comparison



# Semantic top-k operator optimizations:

pairwise quickselect



listwise prompts + fine-tuned rankers

+

multi-pivot optimization

+

tournament filter

---

# **Thank you!**

- burhani fajar kurniawan (lotus clip art)